



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

**Diseño e implementación de un modelo de mantenimiento asistido
por inteligencia artificial para una arquitectura de microservicios en
el sistema PortalAsig**

Trabajo Especial de Grado presentado ante la ilustre
Universidad Central de Venezuela por:
Br. Frank Ángel Ponte Delgado
Tutor: Prof. Walter Hernández

Agradecimientos

*[Placeholder: Escribir aquí los agradecimientos a familiares, amigos, tutor
(Prof. Walter Hernández) y la Ilustre Universidad Central de Venezuela]*

Resumen

[Placeholder: Escribir aquí un resumen formal (máx. 300 palabras). Debe mencionar el problema del monolito legado, la migración a microservicios, la incorporación del modelo autonómico AWARE (PAWARE) para mantenimiento asistido por IA, y la validación exitosa del sistema mediante simulaciones de caos.]

Índice general

Agradecimientos	1
Resumen	2
Introducción	6
1. Planteamiento del problema	8
1.1. Justificación	9
1.2. Objetivo General	9
1.3. Objetivos Específicos	9
2. Marco Teórico	11
2.1. Arquitecturas de Software	11
2.1.1. Arquitecturas Monolíticas	11
2.1.2. Arquitecturas Basadas en Microservicios	11
2.1.3. Comparación entre Arquitecturas Monolíticas y Microservicios	11
2.2. Computación Autónoma	12
2.2.1. Concepto de Computación Autónoma	12
2.2.2. Propiedades de los Sistemas Autónomos	12
2.3. Modelo MAPE-K	12
2.3.1. Componentes del Modelo MAPE-K	12
2.3.2. MAPE-K en Sistemas Distribuidos modernos (AWARE)	13
2.4. Sistemas Auto-adaptativos	13
2.4.1. Definición y Características	13
2.4.2. Relación con la Computación Autónoma	13
2.5. Observabilidad en Sistemas Distribuidos	14
2.5.1. Monitoreo	14
2.5.2. Logs y Métricas	14
2.5.3. Trazabilidad (Tracing)	14
2.6. Mantenimiento de Software	14
2.6.1. Tipos de Mantenimiento	14
2.6.2. Retos en Sistemas Distribuidos	15
2.7. Inteligencia Artificial aplicada al Mantenimiento de Software	15

2.7.1.	Detección de Anomalías	15
2.7.2.	Apoyo a la Toma de Decisiones	15
2.8.	AIOps y Automatización de Operaciones	15
2.8.1.	Concepto de AIOps	15
2.8.2.	Aplicaciones en Sistemas Modernos	16
2.9.	Integración de IA en Arquitecturas de Microservicios	16
2.9.1.	Desafíos de Integración	16
2.9.2.	Oportunidades y Beneficios (Mantenimiento Proactivo)	16
3.	Marco Aplicativo: Servicio PAWARE	17
3.1.	Arquitectura del Ecosistema de Microservicios y Observabilidad	17
3.1.1.	Migración y Componentes del Backend	17
3.1.2.	Stack de Observabilidad	18
3.1.3.	Despliegue y Gestión de Contenedores	18
3.2.	Diseño e Implementación del Servicio PAWARE	19
3.2.1.	Arquitectura Interna de PAWARE	19
3.2.2.	Fase Monitor: Recolección de Datos Operativos	19
3.2.3.	Fase Analyze y Plan: Asistencia con Inteligencia Artificial	19
3.2.4.	Fase Execute: Automatización y Notificación	20
3.2.5.	Fase Knowledge: Registro de Incidentes y Feedback	20
3.2.6.	Interfaz de Gestión de Incidentes (Frontend)	20
4.	Pruebas y Resultados	21
4.1.	Pruebas de Integración y Cobertura de Código	21
4.2.	Automatización End-to-End (E2E)	21
4.3.	Simulación de Caos y Recuperación Asistida	22
5.	Conclusiones y Recomendaciones	23
5.1.	Conclusiones	23
5.2.	Trabajo Futuro y Recomendaciones	24

Índice de figuras

Introducción

En los últimos años, el avance de la inteligencia artificial ha impulsado una transformación significativa en la forma en que se conciben y desarrollan las aplicaciones web. Este fenómeno ha favorecido la transición desde el paradigma de la Web 2.0, centrado en la interacción y la generación de contenido por parte de los usuarios, hacia un modelo más dinámico y adaptativo, directamente asociado con la Web 3.0. En este nuevo contexto, las aplicaciones no solo procesan información, sino que incorporan mecanismos que les permiten analizar su entorno, adaptarse a condiciones cambiantes y evolucionar de manera continua.

En este nuevo escenario, la integración de mecanismos basados en inteligencia artificial surge como una consecuencia natural de esta evolución, permitiendo incorporar capacidades de análisis, automatización y apoyo a la toma de decisiones dentro del sistema.

Para ello, es necesario establecer bases arquitectónicas que faciliten la incorporación progresiva de estos mecanismos sin comprometer la estabilidad del software. Esto implica diseñar aplicaciones que puedan escalar, mantenerse y evolucionar de forma controlada. Sin embargo, la efectividad de esta integración depende en gran medida del estilo arquitectónico, ya que estructuras altamente acopladas pueden limitar la incorporación de nuevos componentes, especialmente en sistemas de mayor complejidad.

Como respuesta a esta limitación, han surgido enfoques arquitectónicos orientados a la modularidad, entre los cuales destacan las arquitecturas basadas en microservicios. Este tipo de arquitectura permite descomponer un sistema complejo en componentes simples e independientes, facilitando el desarrollo incremental, el despliegue continuo y la evolución controlada de sus funcionalidades. Sin embargo, la adopción de estos enfoques también introduce nuevos desafíos, especialmente en lo relacionado con la gestión de la complejidad operativa, la observabilidad del sistema y el mantenimiento en entornos distribuidos.

El sistema PortalAsig, desarrollado para la gestión académica en la Facultad de Ciencias de la Universidad Central de Venezuela, constituye un caso representativo de estas problemáticas. Desde su creación en 2013, el sistema fue concebido con una arquitectura centralizada, acompañada de una interfaz de usuario desarrollada en Angular. Posteriormente, el trabajo de grado de Vásquez Balza [1] introdujo un rediseño del front-end en Vue bajo un enfoque orientado a la mantenibilidad, incorporando prácticas de organización y mejora de la calidad del código.

No obstante, el backend ha permanecido basado en una arquitectura monolítica con limitaciones estructurales significativas, lo que ha incrementado su complejidad y dificultado su evolución. El rediseño del backend hacia una arquitectura de microservicios se plantea como una necesidad para soportar una nueva forma de gestionar el mantenimiento del sistema.

En la actualidad, el mantenimiento del sistema se realiza principalmente de forma reactiva, donde el personal a cargo atiende fallas bajo demanda ante la aparición de errores visibles o durante períodos específicos, como el inicio o cierre de un semestre. Las acciones correctivas son manuales y no responden a un proceso sistemático de diagnóstico o análisis de causa raíz, lo que dificulta la evolución controlada del sistema y la implementación de mejoras.

La integración de mecanismos asistidos por inteligencia artificial representa una oportunidad para transformar el proceso actual de mantenimiento del software. Estas capacidades permiten complementar el trabajo humano mediante la automatización de tareas de análisis, diagnóstico y apoyo en la toma de decisiones, lo cual resulta especialmente relevante en contextos donde los recursos humanos son limitados.

En consecuencia, el presente Trabajo Especial de Grado propone el diseño e implementación de un modelo de mantenimiento asistido por inteligencia artificial para una arquitectura de microservicios en el sistema PortalAsig. Este modelo busca integrar principios de sistemas auto-adaptativos, mecanismos de observabilidad y técnicas de inteligencia artificial, con el objetivo de transformar el mantenimiento del software en un proceso semi-asistido, sistemático, eficiente y alineado con las tendencias de la cada vez más presente Web 3.0.

Capítulo 1

Planteamiento del problema

El sistema PortalAsig, desarrollado para la gestión académica en la Facultad de Ciencias de la Universidad Central de Venezuela, fue concebido como una plataforma centralizada orientada a facilitar la comunicación entre docentes y estudiantes. A lo largo de su evolución, el sistema ha incorporado mejoras parciales, particularmente en su frontend, orientadas a mejorar su organización y mantenibilidad. Sin embargo, su backend ha permanecido basado en una arquitectura monolítica, a pesar de la complejidad del sistema. Esta situación ha dificultado su mantenimiento y su capacidad de adaptación a nuevos requerimientos.

Actualmente, la respuesta ante un error del sistema se detecta de forma empírica y reactiva, principalmente ante fallas visibles o durante períodos específicos de intervención, como el inicio o cierre de un semestre académico. Este mecanismo de mantenimiento del Software depende en gran medida de acciones manuales y del conocimiento implícito de los colaboradores, sin contar con mecanismos formales de monitoreo, observabilidad o diagnóstico que permitan gestionar el sistema de forma eficiente. Como consecuencia, se dificulta la detección temprana de problemas, se incrementan los tiempos de resolución de fallas y se limita la incorporación de nuevas características.

En este escenario, y considerando cómo en pocos años se ha acelerado la transición hacia la Web 3.0, donde los sistemas deben ser capaces de adaptarse y evolucionar de forma continua, se evidencia la necesidad de transformar tanto la arquitectura como el proceso de mantenimiento de PortalAsig. En particular, se identifica la ausencia de un enfoque que combine una arquitectura modular basada en microservicios con mecanismos de observabilidad y técnicas de inteligencia artificial, que permitan asistir el mantenimiento del sistema. Esto permitiría pasar de un esquema enteramente reactivo a uno sistemático y orientado a la mejora rápida y confiable del Software.

1.1. Justificación

La modernización del sistema PortalAsig responde a la necesidad de garantizar su sostenibilidad, mantenibilidad y capacidad de adaptación en el tiempo. En el contexto actual, caracterizado por la transición hacia la Web 3.0, los sistemas de software deben ser capaces de evolucionar de forma continua, lo que exige no solo una arquitectura adecuada, sino también procesos de mantenimiento más eficientes y sistemáticos.

La adopción de una arquitectura basada en microservicios permite establecer una base modular que facilita el aislamiento de cambios y la evolución del sistema. Sin embargo, esta transformación incrementa la complejidad operativa, haciendo necesario incorporar mecanismos que permitan la gestión semi-asistida.

En este sentido, la incorporación de inteligencia artificial como apoyo al mantenimiento del sistema representa una alternativa viable para automatizar tareas de análisis, detección de anomalías y apoyo a la toma de decisiones. Este enfoque permite reducir la dependencia de intervenciones manuales y mejorar la capacidad de respuesta ante fallas, siendo especialmente relevante en el caso de PortalAsig, donde los recursos humanos son limitados.

Por lo tanto, el desarrollo de un modelo de mantenimiento asistido por inteligencia artificial permite no solo mejorar el estado actual del sistema, sino establecer una base para su evolución continua, alineándolo con las exigencias tecnológicas propias de la Web 3.0.

1.2. Objetivo General

Diseñar e implementar un modelo de mantenimiento de software asistido por inteligencia artificial en una arquitectura de microservicios para el sistema PortalAsig, que permita mejorar su mantenibilidad, capacidad de adaptación y sostenibilidad en el tiempo.

1.3. Objetivos Específicos

- Analizar el estado actual del sistema PortalAsig y su proceso de mantenimiento.
- Identificar las limitaciones asociadas a la arquitectura monolítica existente.
- Diseñar una arquitectura basada en microservicios que mejore la modularidad, escalabilidad y mantenibilidad del sistema.
- Diseñar un modelo de mantenimiento basado en principios de sistemas auto-adaptativos.
- Integrar mecanismos de observabilidad para el monitoreo del comportamiento del sistema.

- Implementar un prototipo funcional que incluya el rediseño del backend y el modelo de mantenimiento propuesto.
- Evaluar el impacto del modelo en términos de mantenibilidad, operación y capacidad de adaptación del sistema.

Capítulo 2

Marco Teórico

2.1. Arquitecturas de Software

2.1.1. Arquitecturas Monolíticas

El desarrollo de software tradicional frecuentemente se orienta hacia la construcción de aplicaciones monolíticas. En este tipo de arquitectura, todos los componentes del sistema (interfaz de usuario, lógica de negocio y acceso a datos) se empaquetan y despliegan como una única unidad lógica y ejecutable [2]. Si bien este enfoque simplifica el desarrollo inicial, las pruebas y el despliegue a pequeña escala, tiende a presentar problemas significativos de escalabilidad y mantenimiento a medida que la base de código crece. Un fallo en un componente puede derivar en la caída de toda la aplicación, y actualizar una funcionalidad requiere el redespliegue del sistema entero.

2.1.2. Arquitecturas Basadas en Microservicios

Para mitigar las deficiencias de los monolitos, la industria evolucionó hacia la arquitectura basada en microservicios. Según Fowler y Lewis (2014) [3], los microservicios son un enfoque de desarrollo en el que una aplicación única se construye como una suite de pequeños servicios, donde cada uno ejecuta su propio proceso y se comunica mediante mecanismos ligeros (usualmente APIs HTTP/REST). Estos servicios están contruidos alrededor de capacidades de negocio, son desplegables de manera independiente y pueden ser escritos en diferentes lenguajes de programación o utilizar diferentes tecnologías de almacenamiento de datos.

2.1.3. Comparación entre Arquitecturas Monolíticas y Microservicios

La principal diferencia radica en el acoplamiento y la unidad de despliegue. Las arquitecturas monolíticas ofrecen simplicidad operativa en etapas tempranas

pero sufren de alto acoplamiento. Por el contrario, los microservicios brindan alta cohesión, escalabilidad independiente por servicio, y resiliencia ante fallos aislados. No obstante, introducen una gran complejidad operativa inherente a los sistemas distribuidos, tales como la latencia de red, la tolerancia a fallos, el monitoreo distribuido y la orquestación de despliegues [2].

2.2. Computación Autonómica

2.2.1. Concepto de Computación Autonómica

Inspirada en el funcionamiento del sistema nervioso autónomo del cuerpo humano, la Computación Autonómica fue propuesta por IBM en 2001 para lidiar con la creciente e inmanejable complejidad de la gestión de sistemas informáticos [4]. El concepto define sistemas informáticos capaces de autogestionarse en base a políticas de alto nivel establecidas por administradores humanos, ocultando la complejidad subyacente de las operaciones técnicas.

2.2.2. Propiedades de los Sistemas Autonómicos

Un sistema autonómico se rige típicamente por cuatro propiedades fundamentales (conocidas como las propiedades "Self-CHOP") [5]:

- **Auto-configuración (Self-configuration):** Capacidad de instalar y adaptar componentes de software automáticamente sin intervención externa.
- **Auto-recuperación (Self-healing):** Habilidad para detectar, diagnosticar y reparar problemas operativos, interrupciones y fallos en el software o hardware.
- **Auto-optimización (Self-optimization):** Monitorización proactiva y ajuste continuo de los recursos para garantizar un rendimiento óptimo.
- **Auto-protección (Self-protection):** Capacidad de anticipar, detectar e identificar ataques maliciosos o fallos en cascada, tomando acciones para salvaguardar el sistema.

2.3. Modelo MAPE-K

2.3.1. Componentes del Modelo MAPE-K

El modelo MAPE-K (Monitor, Analyze, Plan, Execute, Knowledge) es el marco arquitectónico de referencia introducido por IBM en 2003 para construir sistemas autonómicos [4]. Divide el proceso de auto-gestión en cuatro fases apoyadas sobre una base de conocimiento compartida:

- **Monitor (Monitoreo):** Recopila información de topología, métricas y logs de los recursos gestionados.

- **Analyze (Análisis):** Procesa la información para entender el estado actual del sistema y determinar si se ha producido una anomalía.
- **Plan (Planificación):** Estructura las acciones necesarias para alcanzar los objetivos o solucionar la falla detectada.
- **Execute (Ejecución):** Aplica el plan a través de actuadores o comandos sobre el sistema objetivo.
- **Knowledge (Conocimiento):** Base de datos o repositorio histórico que nutre y es nutrido por las otras cuatro fases [6].

2.3.2. MAPE-K en Sistemas Distribuidos modernos (AWARE)

La transición hacia microservicios introdujo nuevos niveles de complejidad, haciendo que la implementación estricta de un único ciclo MAPE-K resultara un cuello de botella. Nuevos enfoques, como el marco AWARE (AIOps-based Workflow for Autonomic Resiliency and Execution), adaptan este modelo integrando capacidades de Inteligencia Artificial directamente en las fases de Análisis y Planificación. Esto permite que el sistema pase de decisiones basadas en reglas estáticas a razonamientos basados en modelos de lenguaje (LLMs) u otros clasificadores de aprendizaje automático [7].

2.4. Sistemas Auto-adaptativos

2.4.1. Definición y Características

Un sistema auto-adaptativo tiene la capacidad de modificar su comportamiento y estructura en tiempo de ejecución en respuesta a cambios en su entorno u operaciones. Weyns (2019) establece que un sistema verdaderamente auto-adaptativo requiere retroalimentación constante y mecanismos de toma de decisiones no deterministas que reduzcan la carga cognitiva de los operadores humanos [8].

2.4.2. Relación con la Computación Autónoma

Mientras que la computación autónoma abarca la visión completa de sistemas auto-manejables (auto-reparación, auto-optimización, auto-protección y auto-configuración), la auto-adaptación es el mecanismo de ingeniería de software práctico para implementarlo. Herramientas contemporáneas de aseguramiento de calidad (como análisis de cobertura de *JaCoCo* o pruebas end-to-end con *Cypress*) juegan un rol clave, ya que la auto-adaptación requiere que el código alterado mantenga su validez estructural.

2.5. Observabilidad en Sistemas Distribuidos

2.5.1. Monitoreo

En arquitecturas monolíticas, el monitoreo a menudo se reducía a verificar si el servidor estaba encendido (uptime) y si el uso de CPU/RAM era normal. Sridharan (2018) [9] distingue entre *monitoreo* (el acto de observar el estado general y predecible de un sistema) y la *observabilidad* (la cualidad de un sistema que permite inferir estados internos basándose únicamente en el conocimiento de sus salidas externas). En microservicios, la observabilidad es obligatoria debido a que las fallas rara vez son predecibles y provienen de la interacción compleja entre múltiples componentes pequeños.

2.5.2. Logs y Métricas

Los pilares de la observabilidad se fundamentan en la agregación de telemetría:

- **Métricas:** Valores numéricos agregados a lo largo del tiempo, útiles para disparar alertas cuando se sobrepasan umbrales (ej. latencia p95 o el uso de CPU) [9]. Herramientas como *Prometheus* son el estándar en sistemas modernos.
- **Logs (Registros):** Representan eventos discretos y detallados producidos por las aplicaciones. Al centralizarse en sistemas como *Loki*, los logs proporcionan el contexto exacto (.el qué y el porqué”) cuando una métrica señala que algo anda mal.

2.5.3. Trazabilidad (Tracing)

La trazabilidad distribuida permite el seguimiento del ciclo de vida de una solicitud HTTP única a través de todos los microservicios involucrados en satisfacerla. Aporta un identificador único (Trace ID) que reconstruye la cadena temporal de llamadas de red y tiempos de espera, vital para debugear latencia entre microservicios [9].

2.6. Mantenimiento de Software

2.6.1. Tipos de Mantenimiento

El estándar internacional ISO/IEC/IEEE 14764 [10] establece que el mantenimiento del software ocurre una vez que el sistema ha sido puesto en producción y lo categoriza en cuatro vertientes:

- **Correctivo:** Modificaciones reactivas para corregir fallos o bugs descubiertos.

- **Adaptativo:** Modificaciones para asegurar que el software siga siendo funcional en un entorno tecnológico cambiante (ej. migraciones de base de datos o de S.O.).
- **Perfectivo:** Mejoras en el rendimiento o implementación de nuevas funcionalidades operativas (sin cambiar la lógica de negocio subyacente).
- **Preventivo:** Refactorización o mitigación técnica para prevenir fallos latentes o corregir la deuda técnica antes de que cause incidencias.

2.6.2. Retos en Sistemas Distribuidos

Realizar tareas de mantenimiento en arquitecturas basadas en microservicios introduce retos significativos. La falla no suele concentrarse en una sola línea de código, sino que puede deberse a interrupciones de red, fallos en contenedores terceros o latencias asíncronas [2]. Identificar la raíz del problema (Root Cause Analysis - RCA) consume una porción abrumadora del tiempo del desarrollador, lo cual incrementa el MTTR (Mean Time to Recovery).

2.7. Inteligencia Artificial aplicada al Mantenimiento de Software

2.7.1. Detección de Anomalías

El uso de algoritmos de Machine Learning e IA permite establecer líneas base (baselines) del comportamiento "normal" de un sistema (tasa de errores, latencia habitual, picos estacionales de tráfico). Cuando los patrones reales se desvían de estas líneas base predictivas, la IA puede alertar al equipo antes de que ocurra una interrupción total del servicio [11].

2.7.2. Apoyo a la Toma de Decisiones

El advenimiento de los Modelos de Lenguaje de Gran Escala (LLMs) ha revolucionado la gestión de operaciones de TI. Al abstraer métricas matemáticas, archivos de log crudos y topología de servicios en representaciones de texto, los LLMs son capaces de entender y sintetizar errores complejos en diagnósticos legibles por humanos. Actúan como asistentes (Copilots) para los ingenieros DevOps, sugiriendo fragmentos de código o configuraciones óptimas para recuperar o escalar la infraestructura.

2.8. AIOps y Automatización de Operaciones

2.8.1. Concepto de AIOps

El término AIOps (Artificial Intelligence for IT Operations) fue acuñado por Gartner en 2016 [12] para describir la aplicación de Big Data, Machine Learning

y otras tecnologías avanzadas a los flujos de operaciones de TI. AIOps mejora el monitoreo tradicional al correlacionar eventos ruidosos, encontrar la causa raíz (RCA) con mayor velocidad y proporcionar remediación predictiva.

2.8.2. Aplicaciones en Sistemas Modernos

En ecosistemas orquestados por herramientas como Docker o Kubernetes, AIOps puede predecir cuándo un contenedor fallará analizando un aumento anormal (drift) de la memoria o latencia. La integración de AIOps en modelos autonómicos (como MAPE-K) representa el estado del arte en la ingeniería de resiliencia (Resilience Engineering) [11].

2.9. Integración de IA en Arquitecturas de Microservicios

2.9.1. Desafíos de Integración

La principal barrera para incorporar Inteligencia Artificial en operaciones es la fragmentación de los datos. En un entorno de microservicios, los registros están distribuidos en múltiples nodos. Para que un LLM o un modelo estadístico funcione, se requiere la implementación de patrones de agregación mediante herramientas como Loki y Prometheus, garantizando un punto de entrada centralizado de contexto [13].

2.9.2. Oportunidades y Beneficios (Mantenimiento Proactivo)

Al abstraer las métricas como contexto ("prompts") para modelos generativos, los administradores obtienen diagnósticos en lenguaje natural. Esto democratiza la administración de servidores complejos y acelera drásticamente los tiempos de recuperación (MTTR). Además, permite establecer un canal de auto-mantenimiento donde el sistema puede inferir su propio nivel de riesgo, recomendando a los equipos humanos refactorizar secciones de código o ajustar límites de contenedores antes de que se convierta en una caída masiva.

Capítulo 3

Marco Aplicativo: Servicio PAWARE

Este capítulo introduce el contexto aplicativo del Trabajo Especial de Grado, enfocándose en la infraestructura de microservicios y observabilidad sobre la cual se implementa el modelo de auto-mantenimiento asistido por inteligencia artificial. Se describen los componentes del ecosistema, la estrategia de despliegue y gestión, el diseño e implementación del servicio PAWARE, los escenarios de prueba y los resultados obtenidos.

3.1. Arquitectura del Ecosistema de Microservicios y Observabilidad

Esta sección describe la infraestructura tecnológica sobre la cual se implementará el modelo de mantenimiento de software. Se detalla la migración del backend monolítico de PortalAsig a una arquitectura de microservicios, la incorporación de un robusto stack de observabilidad y el mecanismo de despliegue y gestión del entorno completo, tal como ha sido configurado y validado.

3.1.1. Migración y Componentes del Backend

El sistema PortalAsig, inicialmente concebido como un monolito, ha sido refactorizado hacia una arquitectura de microservicios. Este nuevo ecosistema se compone de múltiples nodos especializados desarrollados en Spring Boot, cada uno con responsabilidades bien definidas:

- **Gestión de Identidad y Acceso:** Microservicio encargado de la autenticación, autorización y emisión de tokens seguros.
- **Gestor de Sitios Académicos:** Maneja la lógica de negocio central, incluyendo la persistencia de cursos, semestres y contenido académico.

- **Servicio de Notificaciones:** Responsable del procesamiento asíncrono y despacho de correos electrónicos.
- **PAWARE:** El servicio de control autónómico que implementa el ciclo de auto-mantenimiento.

[Placeholder: Diagrama de Arquitectura de Microservicios detallando las conexiones y bases de datos por servicio]

La configuración de estos nodos se gestiona a través de un servidor de configuración centralizado (*Config Server*), el cual resuelve propiedades desde un repositorio distribuido. Cada componente persiste sus datos de manera aislada en instancias dedicadas, asegurando alta disponibilidad.

3.1.2. Stack de Observabilidad

Para permitir que PAWARE monitoree el estado del ecosistema, se ha integrado un stack de observabilidad moderno, compuesto por las siguientes herramientas:

- **Prometheus:** Sistema de monitoreo y alertas que recolecta métricas de los microservicios, exponiendo endpoints `/actuator/prometheus`.
- **Loki:** Sistema de agregación de logs diseñado para almacenar y consultar grandes volúmenes de logs de manera eficiente.
- **Grafana:** Plataforma de visualización que permite crear dashboards interactivos para explorar métricas de Prometheus y logs de Loki, proporcionando una visión consolidada del estado del sistema.

Estos componentes se configuran para interactuar entre sí, permitiendo una recolección exhaustiva de datos sobre el comportamiento del sistema.

3.1.3. Despliegue y Gestión de Contenedores

El ecosistema completo se orquesta de manera estandarizada empleando *Docker Compose*. Mediante archivos de definición declarativos, se despliegan simultáneamente las bases de datos, los componentes de observabilidad y las instancias de los microservicios. Este enfoque asegura que todas las dependencias compartan una única red virtual, permitiendo el enrutamiento interno mediante resolución de nombres nativa de Docker.

[Placeholder: Fragmento del archivo docker-compose.yml o captura de consola demostrando la orquestación y el estado de los contenedores]

Para optimizar los flujos de despliegue en entornos de desarrollo local, se implementaron scripts de terminal capaces de inicializar bases de datos, detener entornos y visualizar flujos de logs centralizados sin requerir configuración manual extensa.

3.2. Diseño e Implementación del Servicio PAWARE

Esta sección detalla el diseño arquitectónico y la implementación del servicio PAWARE, el componente central que orquesta el ciclo de auto-mantenimiento asistido por inteligencia artificial en el ecosistema de PortalAsig. Se describe cómo PAWARE monitorea, analiza, planifica y ejecuta acciones, delegando las tareas de análisis y planificación a un modelo de IA externo.

3.2.1. Arquitectura Interna de PAWARE

PAWARE opera como un microservicio independiente que encapsula físicamente la estructura del ciclo MAPE-K. Su arquitectura interna se basa en procesos asíncronos programados que evalúan periódicamente la estabilidad del ecosistema. Se divide en módulos lógicos: conectores de observabilidad, un orquestador del lazo de control autónomo y un adaptador para la Inteligencia Artificial. Asimismo, PAWARE expone puntos de acceso (API REST) consumidos por las interfaces de usuario para permitir la auditoría y la interacción manual cuando se requiere supervisión.

3.2.2. Fase Monitor: Recolección de Datos Operativos

Durante la monitorización, el sistema centraliza la telemetría operativa. Efectúa consultas a las plataformas de agregación de métricas (Prometheus) para evaluar el estado de salud, carga de la infraestructura y tasas de respuesta. Simultáneamente, consulta el motor de registros (Loki) para consolidar excepciones, errores en tiempo de ejecución o advertencias críticas. Los enlaces de conexión hacia estos monitores se inyectan a través de variables de entorno, dotando a la herramienta de total portabilidad e independencia de infraestructura específica.

3.2.3. Fase Analyze y Plan: Asistencia con Inteligencia Artificial

Una vez que la fase Monitor consolida las métricas y logs en un `MonitoringSnapshot`, PAWARE transiciona a las fases de Análisis y Planificación. Si se detectan anomalías —tales como servicios caídos (`DOWN`), latencias superiores a los umbrales esperados en métricas de Prometheus, o un alto volumen de excepciones detectadas mediante Loki— el snapshot es procesado por un proveedor de Inteligencia Artificial. El sistema abstrae la integración mediante la interfaz `AwareAiProvider`, permitiendo el uso de modelos como OpenAI o Gemini. El LLM evalúa el snapshot, diagnostica la raíz del problema y estructura un plan de acción en formato JSON. Este plan asigna una prioridad (P1, P2 o P3), sugiere un tipo de acción (`SEMI_ASSISTED` o `AUTONOMOUS`) y propone estrategias de remediación como reiniciar el servicio, escalar infraestructura o revisar el código fuente.

3.2.4. Fase Execute: Automatización y Notificación

En la fase de ejecución, PAWARE procesa las acciones recomendadas por el modelo de IA. Las acciones pueden ser de dos tipos principales:

- **Acciones Automáticas:** PAWARE puede ejecutar comandos directamente sobre el entorno Docker. Por ejemplo, para un reinicio de servicio (AUTOMATED_RESTART), invoca el script `portalsig-cli` con el comando `restart` para el microservicio objetivo.
- **Notificaciones:** Para acciones que requieren intervención humana o simplemente informar, PAWARE interactúa con el microservicio `ms-notify` para enviar alertas por correo electrónico a los administradores del sistema, incluyendo detalles del incidente y las recomendaciones de la IA.

Cada acción, ya sea automática o de notificación, es registrada en la base de datos de PAWARE, manteniendo un historial completo.

3.2.5. Fase Knowledge: Registro de Incidentes y Feedback

Todo el estado, los planes propuestos por la IA y la evidencia de las fallas se almacenan en una base de datos relacional (tablas `paware.alert` y `paware.alert.feedback`). Esta persistencia actúa como la base de Conocimiento (Knowledge) del modelo MAPE-K. Los administradores interactúan con estas alertas para aprobar o rechazar acciones. Estas decisiones humanas, acompañadas de comentarios, se retroalimentan al sistema. A futuro, este registro continuo de incidentes y decisiones permitirá perfeccionar los prompts o realizar fine-tuning del modelo de IA, logrando un proceso de auto-mantenimiento más autónomo y robusto.

3.2.6. Interfaz de Gestión de Incidentes (Frontend)

Se incorporó un módulo dedicado denominado “Alertas PAWARE” dentro de la aplicación frontend en Vue.js. Esta vista presenta de manera resiliente las alertas activas, divididas en “Pendientes de Acción” y “Completadas / Gestionadas”. Para facilitar la atención del administrador, las incidencias se ordenan por severidad y se destacan con códigos de color (rojo para P1, amarillo para P2).

Además, se integró un Asistente Virtual basado en LLM. Los administradores pueden abrir un chat contextual para una alerta específica y consultar a la IA sobre la naturaleza del error o los pasos manuales sugeridos, emulando una experiencia de operador asistido o “Copilot” de infraestructura.

[Placeholder: Captura del listado de alertas priorizadas en el Frontend]

[Placeholder: Captura del Chat con el asistente de IA en la vista de Alertas]

Capítulo 4

Pruebas y Resultados

Para validar la confiabilidad tanto del nuevo ecosistema de microservicios como de las capacidades autonómicas del servicio de auto-mantenimiento, se definió una estrategia rigurosa de aseguramiento de calidad. Este capítulo expone las herramientas utilizadas, los escenarios de simulación de caos ejecutados y los resultados obtenidos en el sistema.

4.1. Pruebas de Integración y Cobertura de Código

En el backend, se priorizó la ejecución de pruebas de integración reales en lugar de pruebas aisladas. Para ello, se emplearon contenedores efímeros mediante la librería *Testcontainers*, lo que permitió instanciar bases de datos relacionales y servicios de agregación de bitácoras durante la fase de prueba. Esto garantizó que el sistema se evaluara sobre un entorno idéntico al de producción.

Para cuantificar el nivel de fiabilidad, la cobertura de código se validó mediante el uso de *JaCoCo* (Java Code Coverage). Se obtuvieron métricas de cobertura general que demostraron que las rutas críticas, especialmente las asociadas a la orquestación del ciclo de auto-mantenimiento, se encontraban robustamente cubiertas.

[Placeholder: Captura del reporte HTML de JaCoCo mostrando la cobertura del servicio de auto-mantenimiento]

4.2. Automatización End-to-End (E2E)

Para la validación del sistema desde la perspectiva del usuario final, se evaluaron diversas herramientas de automatización de pruebas de interfaz (Frontend). Si bien herramientas consolidadas basadas en controladores externos (*WebDrivers*) son un estándar, suelen introducir inestabilidad (*flakiness*) y tiempos de ejecución prolongados.

Se seleccionó *Cypress* como herramienta definitiva debido a que se ejecuta en el mismo ciclo de vida del navegador y del marco de trabajo del Frontend, lo que incrementa notablemente la velocidad y estabilidad de las pruebas. Dado que el ecosistema descentralizado abstrae la validación de identidad en un servicio especializado (UAA), las pruebas se diseñaron para realizar peticiones directas de inicio de sesión e inyectar de forma transparente los tokens resultantes, mitigando así la complejidad de navegar múltiples dominios durante la prueba.

[Placeholder: Captura de la ejecución exitosa de pruebas Cypress en consola o en su interfaz gráfica]

4.3. Simulación de Caos y Recuperación Asistida

Para comprobar la resiliencia del ecosistema y el funcionamiento del modelo de auto-mantenimiento, se diseñó un escenario de prueba basado en ingeniería del caos. Se detuvo intencionalmente la ejecución del componente encargado de la lógica académica (el servicio de sitios), emulando una caída catastrófica en un entorno de producción.

El comportamiento registrado fue el siguiente:

1. **Detección y Aislamiento:** En su siguiente ciclo de monitorización, el orquestador autonómico detectó la caída del servicio y los errores subsecuentes reportados por el sistema de recolección de métricas. Gracias a que el Frontend fue rediseñado de manera tolerante a fallos, la aplicación web absorbió el error de red y continuó operando de forma parcial, aislando la caída.
2. **Alerta y Notificación:** El servicio autonómico analizó la situación, categorizando la eventualidad como un incidente de máxima prioridad (P1). Inmediatamente, delegó al servicio de notificaciones la tarea de despachar un correo de advertencia con formato institucional.
3. **Asistencia y Decisión:** El administrador fue capaz de ingresar al portal, dirigirse al módulo de incidentes y visualizar el evento. La interfaz, además de presentar las evidencias recopiladas, ofreció un chat inteligente que permitió al administrador solicitar a la IA la descripción detallada del error y confirmar la acción de auto-reparación.

Las siguientes figuras ilustran la recepción efectiva de la alerta y la interfaz provista al operador para gestionar la resolución del problema:

[Placeholder: Captura del correo de alerta (P1) generado por el servicio de notificaciones, visible en la bandeja de entrada]

[Placeholder: Captura de la alerta P1 generada en el Frontend evidenciando la caída del servicio, acompañada del chat de asistencia]

Capítulo 5

Conclusiones y Recomendaciones

5.1. Conclusiones

La evolución del sistema de gestión académica hacia una arquitectura orientada a microservicios permitió resolver satisfactoriamente las carencias de escalabilidad, acoplamiento y despliegue que afectaban al monolito original. La segregación de responsabilidades en dominios independientes —tales como identidad, lógica académica y notificaciones— simplificó la comprensión del sistema, redujo el riesgo de fallas en cascada y permitió adoptar tecnologías modernas adaptadas a las necesidades específicas de cada módulo.

En paralelo, la incorporación de un modelo de auto-mantenimiento demostró ser una solución pragmática e innovadora frente a la escasez de recursos humanos dedicados a la administración operativa. El sistema desarrollado no solo logró consolidar las métricas de rendimiento y los registros de error dispersos, sino que fue capaz de inferir proactivamente soluciones mediante la integración de capacidades de Inteligencia Artificial (LLMs). Las pruebas de caos demostraron que el sistema es capaz de detectar la caída de servicios, alertar inmediatamente a las partes interesadas mediante notificaciones institucionalizadas y proveer al operador un entorno de diagnóstico asistido e inmediato.

En definitiva, se comprobó empíricamente que la observabilidad distribuida, combinada con algoritmos generativos orientados a infraestructura (AIOps), acelera drásticamente el diagnóstico de fallas (Root Cause Analysis), reduciendo los tiempos de recuperación (MTTR) e incrementando la disponibilidad del sistema.

5.2. Trabajo Futuro y Recomendaciones

Para potenciar el alcance y la autonomía del modelo implementado, se sugieren las siguientes vías de desarrollo futuro. La visión a largo plazo es dotar a PAWARE de un contexto mucho más profundo y herramientas de acción avanzadas, evolucionándolo de un asistente de diagnóstico a un agente de ingeniería de software completamente autónomo:

- **Integración Profunda con el Código Fuente (Control de Versiones):** Ampliar las capacidades de la IA para que pueda conectarse a los repositorios del proyecto (ej. GitHub/GitLab). Esto le permitiría, ante una alerta, clonar el código, auditar la lógica, encontrar el segmento defectuoso y sugerir —o incluso generar automáticamente— *Pull Requests* con la refactorización o corrección (auto-mejora del código).
- **Interacción Activa con Servicios y Datos:** Evolucionar a PAWARE para que no solo consuma métricas pasivas de observabilidad, sino que pueda ejecutar consultas a través de las APIs internas o inspeccionar el estado de las bases de datos de forma segura. Si un operador consulta: “¿Cómo podemos mejorar el rendimiento del módulo de asignaturas?”, la IA podría contrastar la telemetría con los volúmenes de datos reales y emitir recomendaciones de índices de base de datos o rediseño de queries.
- **Tareas Programadas y Auditorías Predictivas:** Habilitar a PAWARE para ejecutar ciclos de revisión preventiva programados (cron jobs de auditoría). En lugar de reaccionar a incidentes, la IA simularía flujos de carga internamente, evaluaría la deuda técnica o los cuellos de botella latentes, y emitiría informes periódicos de mejora continua antes de que los usuarios experimenten lentitud.
- **Ejecución Autónoma en Orquestadores:** Avanzar del estado de recomendación.^a la acción directa sobre la infraestructura (Kubernetes o Docker Swarm), otorgando permisos controlados a la IA para reiniciar, apagar o escalar instancias horizontalmente sin confirmación humana explícita, completando la visión de un sistema auto-reparable (Self-healing).
- **Modelos Específicos y Soberanía Tecnológica (SLMs):** Recopilar progresivamente las resoluciones aprobadas en la base de datos de conocimiento para entrenar modelos de lenguaje de menor escala (Small Language Models). Esto lograría que el asistente capture la terminología exclusiva de la Universidad Central de Venezuela y opere localmente, reduciendo costos y dependencia de APIs de terceros.

Referencias

- [1] R. A. Vásquez Balza, “Implementación de una metodología de mantenimiento de software para la solución y prevención de fallas del sistema portalesig2,” 2020. Trabajo Especial de Grado, Universidad Central de Venezuela.
- [2] C. Richardson, *Microservices patterns: with examples in Java*. Simon and Schuster, 2018.
- [3] M. Fowler and J. Lewis, “Microservices: a definition of this new architectural term,” *Martin Fowler*, vol. 14, 2014.
- [4] J. O. Kephart and D. M. Chess, “The vision of autonomic computing,” *Computer*, vol. 36, no. 1, pp. 41–50, 2003.
- [5] N. C. Mendonça *et al.*, “An architecture for autonomic control of software systems,” *Journal of Systems and Software*, vol. 85, no. 12, pp. 2794–2811, 2012.
- [6] IBM, “An architectural blueprint for autonomic computing,” *IBM White Paper*, 2006.
- [7] Y. Dang, Q. Lin, and P. Huang, “Aiopts: real-world challenges and research innovations,” in *2019 IEEE/ACM 41st International Conference on Software Engineering: Companion (ICSE-Companion)*, pp. 4–5, IEEE, 2019.
- [8] D. Weyns, *Software Engineering of Self-Adaptive Systems*. Springer, 2019.
- [9] C. Sridharan, *Distributed Systems Observability*. O’Reilly Media, Inc., 2018.
- [10] IEEE, “Iso/iec/ieee international standard for software engineering – software life cycle processes – maintenance,” 2006.
- [11] P. Notaro *et al.*, “A survey of aiops methods for failure management,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 30, no. 4, pp. 1–45, 2021.
- [12] Gartner, “Market guide for aiops platforms,” tech. rep., Gartner Research, 2017.

- [13] S. Newman, *Building microservices: designing fine-grained systems*. O'Reilly Media, Inc., 2015.